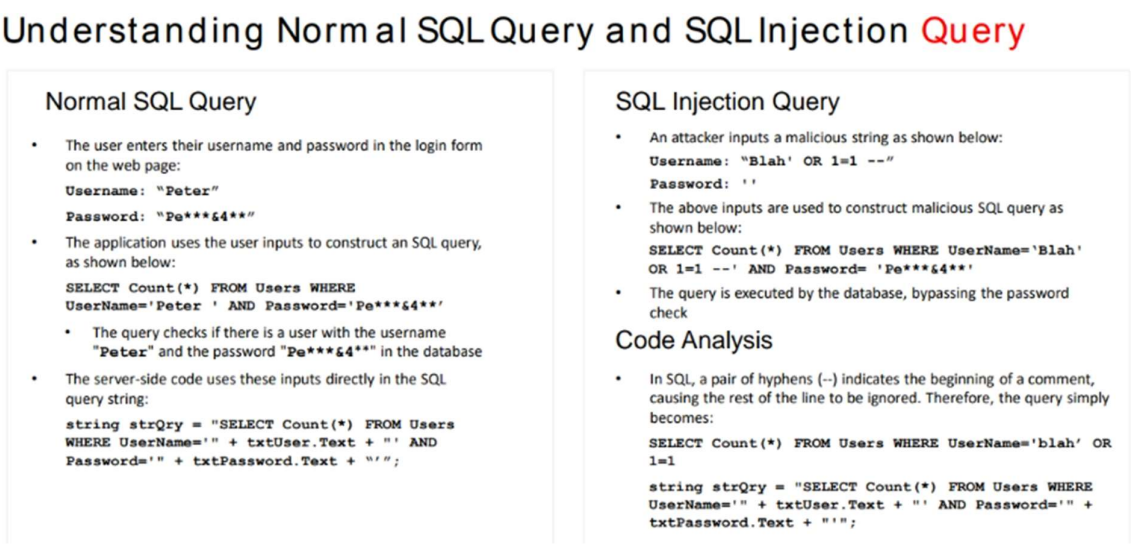
What is SQL Inject ion?
• SQL injection is a technique used to take advantage of un-sanitized input vulnerabilities to pass SQL commands through a web application for execution by a backend database
• SQL injection is a basic attack used to either gain unauthorized access to a database or retrieve information directly from the database •
It is a flaw in web applications and not a database or web server issue

Why Bother About SQL Injection?
Based on the use of applications and the way they process user supplied data, SQL injections can be used to implement the following types of attacks:

1. Authentication and Authorization Bypass
2. Information Disclosure
3. Compromised Integrity and Availability of Data
4. Remote Code Execution



2. Various Types of SQL Injection Attacks

SQL injection attacks vary based on how data is retrieved or how the attack impacts the application.

In-band SQL Injection

Attackers use the same communication channel to perform the attack and retrieve the results. This is the most common and easiest type to exploit.

- **Error-based SQL Injection:** Attacker intentionally triggers database error messages to reveal sensitive information about the database structure (e.g., table names, column names).
- **UNION SQL Injection:** Uses the UNION SELECT statement to combine the results of a malicious query with a legitimate query, allowing attackers to retrieve data from other tables or databases.
- **System Stored Procedure:** Exploiting vulnerabilities in stored procedures to gain unauthorized access or perform malicious actions.
- **Illegal/Logically Incorrect Query:** Injecting logically incorrect statements to deduce information from error messages or application behavior.

- **Tautology:** Injecting conditions that are always true (e.g., OR 1=1), bypassing authentication.
- **End-of-Line Comment:** Using comment characters (-- or #) to ignore the rest of the original SQL query, making the injected query execute.
- **Inline Comment:** Similar to end-of-line comments, but comments are inserted within the query.
- **Piggybacked Query:** Injecting additional SQL queries (separated by a semicolon ;) that are executed alongside the original query. This can lead to data manipulation, DoS, or remote code execution.

Blind/Inferential SQL Injection

Attackers do not receive direct feedback from the database. They infer information by observing application behavior, response times, or generic error messages. This type of attack is often time-consuming.

- **Boolean-based Blind SQL Injection:** Sends SQL queries that return a TRUE or FALSE result. Attackers infer information by observing whether the application's response changes (e.g., page content, error message).
- **Time-based Blind SQL Injection:** Sends SQL queries that cause the database to delay its response based on the result of a logical condition (e.g., IF(condition, BENCHMARK(delay, 1))). Attackers infer information by measuring the time taken for the response.
- **Heavy Query:** Performing computationally intensive queries to consume database resources and observe delays for inference.
- **Double Blind SQL Injection:** A more sophisticated time-based blind SQL injection that doesn't provide direct feedback or error messages.

Out-of-band SQL Injection

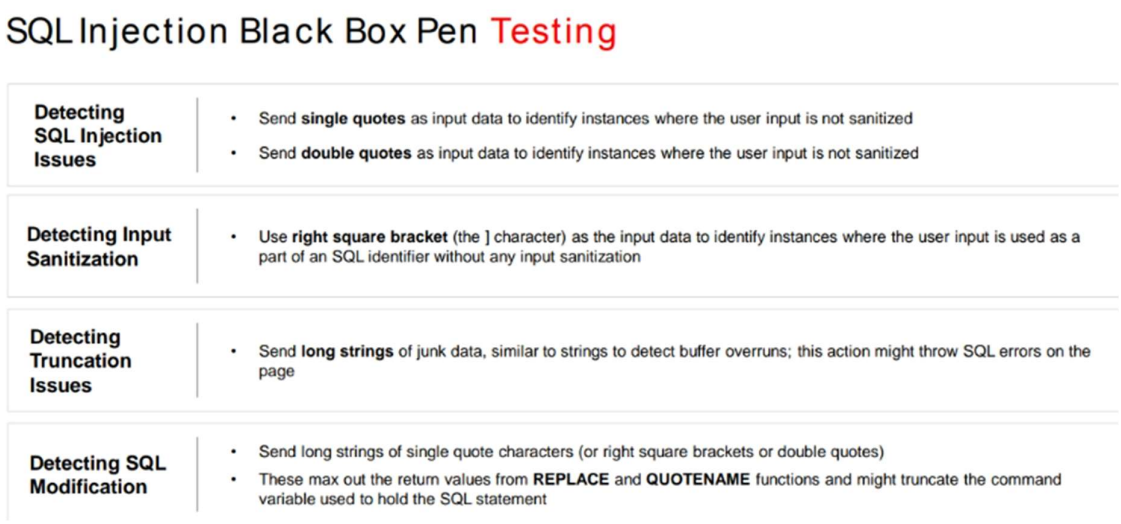
Attackers use different communication channels (e.g., DNS, HTTP, SMB) to send query results or perform file writing functions. This is useful when direct in-band communication is blocked.

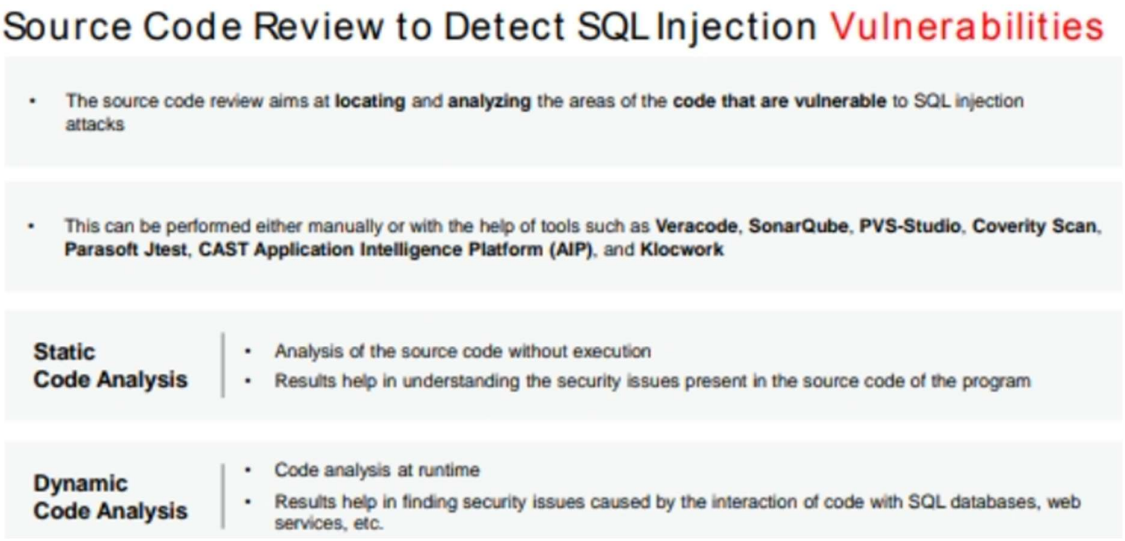
3. SQL Injection Methodology

A systematic approach used by attackers to compromise a web application via SQL injection.

1. **Information Gathering and SQL Injection Vulnerability Detection:**
    - **Target Information:** Gather details about the target database (name, version, users, privileges, OS).
    - **Identify Data Entry Paths:** Find all input fields (GET/POST requests, hidden fields, cookies) that interact with the database. Tools: Burp Suite, Tamper Dev.
    - **Extract Information through Error Messages:** Attackers exploit **error messages** to extract information about the database type, version, structure, and server configuration. Techniques include **parameter tampering**, **ODBC error analysis**, **type mismatch**, and **GROUP BY errors** to trigger detailed responses that reveal backend system details.

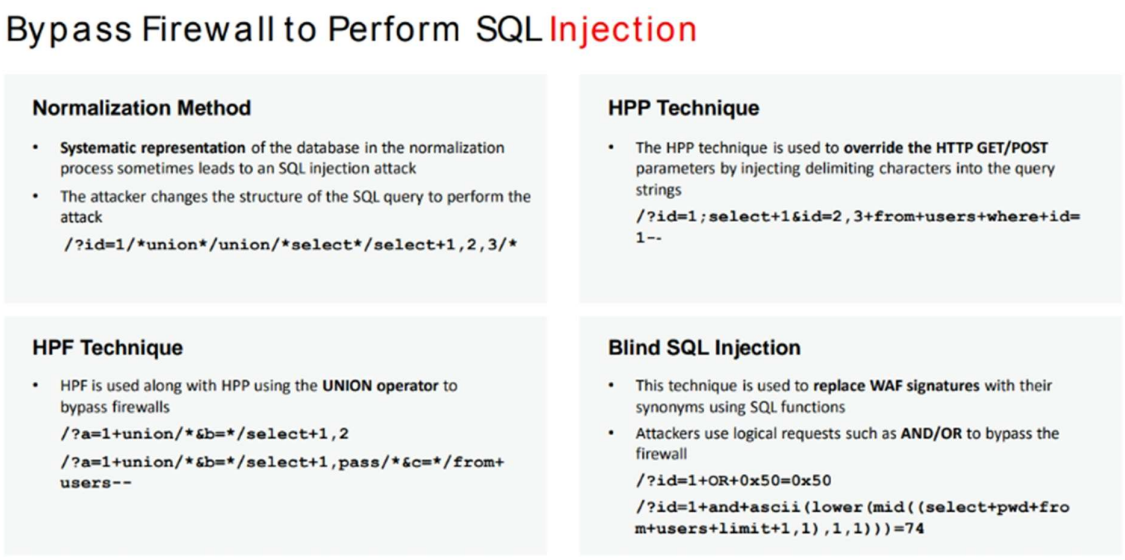
- **SQL Injection Vulnerability Detection:** Test various SQL injection strings to see if the application is vulnerable.
- **Source Code Review:** Analyze application source code (manual or using SAST tools like Veracode, SonarQube, Fortify) for SQL injection vulnerabilities.
- **Function Testing (Black Box):** Send crafted inputs to observe application behavior.
- **Fuzz Testing (Black Box):** Send large amounts of random data to discover coding errors and security loopholes. Tools: BeSTORM, Burp Suite, AppScan Standard.
- **SQL Injection Black Box Pen Testing:** Systematically test input fields for SQL injection issues (detecting input sanitization, truncation, SQL modification).
- **Detecting SQL Modification:** Observe how long strings of single quotes or brackets affect the query.

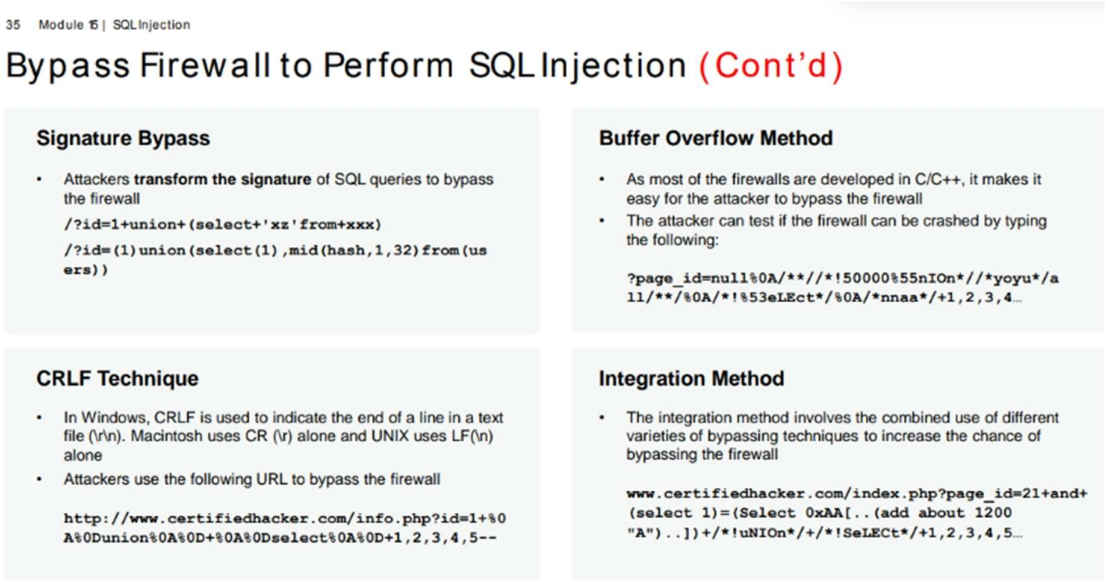




2. **Launch SQL Injection Attacks:** Once vulnerabilities are detected, the attacker proceeds with exploitation.

- **Perform Error Based SQL Injection:** Exploit error messages for data extraction. extract database name, tables, column names, 1st field data
- **Perform Error Based SQL Inject ion using Stored Procedure Inject ion**: When using dynamic SQL within a stored procedure, the application must properly sanitize the user input to eliminate the risk of code injection, otherwise there is a chance of malicious SQL being executed within the stored procedure
- **Perform UNION SQL Injection:** Use UNION SELECT to retrieve data from other tables/columns.
- **Perform Blind SQL Injection:** Use Boolean or time-based techniques to infer information character by character.
- **Perform Out-of-Band SQL Injection:** Exfiltrate data via alternative channels (e.g., DNS).
- **Exploiting Second-Order SQL Injection:** Attacks where the injected payload from a previous request is stored and then executed in a later, different operation.





3. **Advanced SQL Injection (Compromising the Entire Network):**
   - **Database, Table, and Column Enumeration** allows attackers to extract database structure and escalate access. They use SQL queries to:
     - Identify user privileges with functions like user(), session_user, or delays (waitfor delay) to confirm admin access.
     - **Discover admin accounts** (e.g., sa, root, dbo) to perform elevated operations.
     - **Enumerate DB structure** by probing table/column names (sysobjects, syscolumns, all_tab_columns) using UNION, GROUP BY, or system-specific queries across DBMSs like MySQL, MSSQL, Oracle, PostgreSQL, etc
   - **Advanced Enumeration** involves deeper probing to gather system-level and network-level details:
     - Combined Table/Column Enumeration: Attackers join sys.objects, sys.columns, and sys.types to list all table structures in a single query.
     - Database Discovery: They extract names and file paths of all databases using queries on master.dbo.sys.databases.
     - Password Grabbing: By manipulating variables and temporary tables, attackers extract usernames and passwords from the users table, storing them for further exploitation.
   - **Grabbing SQL Server Hashes**
     - **Hash Extraction:** Use SELECT name, password FROM sys.syslogins to extract login hashes (requires DBA access).
     - **Hex Conversion:** Convert binary hash to hexadecimal using SQL logic with loops and character mapping.

- **Exfiltration via Errors:** Hashes can be leaked through crafted error messages using SUBSTRING() and SELECT operations, e.g.,' and 1 in (select substring(x, 256, 256) from temp) –
- **Limited Privilege Scenario:**If DBA access is unavailable, attackers may still enumerate usernames and attempt brute-force attacks on passwords.

- **Transfer Database to At tacker's Machine**
  An SQL Server can be linked back to an attacker's DB via OPENROWSET. The DB Structure is replicated, and the data is transferred. This can be accomplished by connecting to a remote machine on port 80

- **Interact ing with the Operat ing System**
  There are two ways to interact with an OS:
  • Reading and writing system files from the disk
  • Direct command execution via remote shell

- Interacting with the File System
  - The LOAD_FILE() function within MySQL is used to read and return the contents of a file located within the MySQL server
  - The OUTFILE() function within MySQL is often used to run a query and dump the results into a file
- **Network Reconnaissance Using SQL Injection:** Using SQL queries to perform network reconnaissance (e.g., xp_cmdshell with ping, nmap, netstat).
- **Finding and Bypassing Admin Panel:** Using Google Dorks or SQL injection to find and access hidden admin login pages.
- PL/SQL Exploitation: Gaining control over PL/SQL procedures.
- **Creating Server Backdoors:** Writing web shells or executing OS commands.
- **HTTP Header-Based SQL Injection:** Injecting through HTTP headers.
- **DNS Exfiltration using SQL Injection:** Exfiltrating data via DNS queries.
- **MongoDB/NoSQL Injection:** Targeting NoSQL databases.
- **Grabbing SQL Server Hashes:** Extracting password hashes from database system tables.
  - **Transfer Database to Attacker's Machine:** Using OPENROWSET (MSSQL) or other methods to transfer entire databases.
  - **Interacting with the Operating System:** Executing OS commands via xp_cmdshell (MSSQL) or sys_exec/sys_eval (MySQL).
  - Bypassing Website Logins using SQL Injection: Using OR 1=1 or similar techniques to bypass login forms.

SQL Inject ion Tools
sqlmap automates the process of detecting and exploiting SQL injection flaws and the taking over of database servers
Mole is an SQL injection exploitation tool that detects the injection and exploits it only by providing a vulnerable URL and a valid string on the site

4. SQL Injection Evasion Techniques

Attackers use evasion techniques to obscure input strings to avoid detection by signature-based detection systems
Attacker
• Signature-based detection systems build a database of SQL injection attack strings (signatures) and then compare input strings against the signature database during runtime to detect attacks
Types of Signature Evasion Techniques Different types of signature evasion techniques are listed below:

- In-line Comment: Obscures input strings by inserting in-line comments between SQL keywords.
- Char Encoding: Uses a built-in CHAR function to represent a character. ▪ String
- Concatenation: Concatenates text to create an SQL keyword using DB-specific instructions.
- Obfuscated Code: Obfuscated code is an SQL statement that has been made difficult to understand.
- Manipulating White Spaces: Obscures input strings by inserting a white space between SQL keywords.
- Hex Encoding: Uses hexadecimal encoding to represent an SQL query string.
- Sophisticated Matches: Uses alternative expression of ”OR 1=1”.
- URL Encoding: Obscures an input string by adding the percent sign (%) before each code point.
- Null Byte: Uses the null byte (%00) character prior to a string to bypass the detection mechanism.
- Case Variation: Obfuscates SQL statement by mixing it with upper and lowercase letters.
- Declare Variables: Uses variables to pass a series of specially crafted SQL statements and bypass the detection mechanism.
- IP Fragmentation: Uses packet fragments to obscure the attack payload, which goes undetected by the signature mechanism.
- Variations: Uses a WHERE statement that is always evaluated as “true”, so that any mathematical or string comparison can be us

5. SQL Injection Countermeasures

Protecting against SQL injection is primarily about rigorous input validation and secure coding practices.

Web Application Countermeasures

- **Input Validation:** The most crucial defense. Validate all user input (data type, length, format, characters) before processing.
    - **Whitelist Validation:** Allow only known good input.

    - **Blacklist Validation:** Block known bad input (less effective).
- **Use Prepared Statements with Parameterized Queries:** This is the most effective defense. SQL code is defined first, and then parameters are passed separately. The database engine treats parameters as data, not executable code, preventing injection.
- **Stored Procedures:** Use stored procedures that do not concatenate user input directly into SQL queries. Input parameters should be explicitly defined and validated.
- **Principle of Least Privilege:** Grant database users only the minimum necessary permissions. The web application's database account should only have access to specific data required by the application.
- **Disable Default Database Accounts:** Remove or disable default/sample database accounts.
- **Strong Password Policies:** Enforce complex and regularly changed passwords for database accounts.
- **Error Handling:** Implement custom, generic error messages. Do not display detailed database error messages to users, as these can reveal sensitive information.
- **Regular Patching and Updates:** Keep database servers, web servers, and application frameworks patched.
- **Database Security Audits:** Regularly audit database configurations and security settings.
- **Web Application Firewall (WAF):** Deploy a WAF to filter malicious web traffic and block common SQL injection patterns.
- **IDS/IPS:** Use Intrusion Detection/Prevention Systems to monitor and alert on suspicious SQL injection attempts.
- **Encrypt Data:** Encrypt sensitive data in the database.
- **Database Activity Monitoring (DAM):** Monitor real-time database activity for suspicious queries.
- **Limit Network Access:** Restrict database access only to necessary application servers.
- **Use Secure Coding Best Practices:** Train developers on secure coding standards (e.g., OWASP Secure Coding Principles).
- **Web Application Security Testing:** Regularly perform penetration testing, vulnerability scanning (SAST/DAST), and code reviews.
- **Configure Database Specific Hardening Settings:** Implement specific security features for MySQL, MS SQL, Oracle, etc.
- **Monitor for SQLi Detection Tools:** Monitor for the presence or use of SQLi detection tools against the application.

SQL Injection Detection Tools

- **sqlmap:** Open-source penetration testing tool that automates the process of detecting and exploiting SQL injection flaws. Supports1 various types of SQLi (blind, error-based, union, out-of-band).

- **Mole:** Automated SQL injection exploitation tool, supports MySQL, PostgreSQL, SQL Server, and Oracle.
- **Havij:** Automated SQL injection tool.
- **NoSQLMap:** Tool for NoSQL database enumeration and injection.
- **Metasploit:** Can be used for SQL injection exploits.
- **Burp Suite:** Web application testing tool with intruder/repeater for manual SQLi testing.
- **OWASP ZAP:** Open-source web application security scanner.
- **BeSTORM:** Fuzzing tool for finding SQL injection vulnerabilities.
- **AppScan Standard:** Commercial web application security scanner.
- **Static Code Analysis Tools (SAST):** Veracode, SonarQube, PV-Studio, Coverity Scan, Fortify.
- **Dynamic Analysis Tools (DAST):** Invicti, Acunetix, IBM AppScan.

Discovering SQL Injection Vulnerabilities with AI

Attackers can leverage AI-powered technologies (e.g., ChatGPT combined with sqlmap) to automate network-scanning tasks and effortlessly perform SQL injections.